

Progress Report

Sprint 3

Beyza Ünlü, Niklas Zoz, Denis Maxheimer

September 17, 2026

Contents

1	Introduction	2
2	Framework	2
2.1	Data Profiling	2
2.2	Iterative Calibration	3
2.3	Generation Strategies	3
2.4	Module decoupling	4
3	Tasks	4
3.1	Sentiment Analysis	5
3.2	Taxonomy Induction	5
4	Ablation	6
4.1	Setup	6
4.2	Results	7
5	Analysis	9
6	Conclusion	11
	References	12

1 Introduction

Earlier sprints established the framework’s task-based architecture - a common pipeline that loads a benchmark, dispatches generation and evaluation to a pluggable task implementation, and compares the result against a real-data baseline and used it to implement Grammatical Error Correction (Yannakoudakis et al. 2011; Bryant et al. 2019) and Spam Detection (Almeida et al. 2011).

This sprint, the third and final one, focuses on making that architecture general and mature enough to support a wider range of tasks and a systematic evaluation of the framework’s own design choices. Section 2 describes four extensions: richer benchmark profiling, a calibration phase that corrects generation distributions before a scored run, a formal account of the framework’s four generation cells, and the decoupling of the pipeline into independently runnable stages. Section 3 describes two new tasks built on this architecture, Sentiment Analysis and Taxonomy Induction. Section 4 presents the design of a four-phase ablation study that uses these extensions to identify, for every task, a well-justified default configuration.

2 Framework

This sprint’s framework work extends benchmark profiling, formalises calibration as a distinct pre-run phase, gives the generation-cell design a single consistent account across tasks, and decouples the pipeline into independently runnable stages.

2.1 Data Profiling

The framework distinguishes two kinds of profile that are easy to conflate: a *benchmark profile*, built once from the real benchmark and consumed as an input to seedless generation, and a *fidelity profile*, built during or after a run to compare the generated data against the real benchmark. This sprint extended the benchmark profile with samplable characteristics rather than descriptive statistics alone: length is captured as histograms over canonical word- and character-count bins rather than a single mean, alongside style rates and vocabulary statistics, so that seedless generation can sample a length/style content specification instead of reproducing the benchmark’s average example. For GEC, the profile additionally includes syntactic complexity computed with spaCy, and fidelity comparison now also reports a length divergence alongside style deltas.

On top of these descriptive statistics, profiling gained an opt-in LLM-based topic model (`profile_dataset --topics`), using a two-pass open-labelling design: candidate topics are proposed from a sample of the benchmark and then consolidated into a small set of canonical topics that every item is assigned to. A live micro-run on the FCE corpus (20 samples) produced nine coherent topics (e.g. architecture, accommodation, transport, student life). Because seedless generation samples a topic alongside length and style to build its content specification, this is what lets the seedless cells described in Section 2.3 invent on-topic examples without ever using a real one.

Profiling was also generalised to the two tasks added this sprint (Section 3). Sentiment Analysis reuses the same length/style/topic profile as GEC and Spam Detection. Taxonomy Induction instead uses a structural profile - class count, root and leaf counts, hierarchy depth, and parent-count and child-count distributions since a synthetic taxonomy has no length or topic in the text-profiling sense; topic extraction is consequently not applicable to it.

2.2 Iterative Calibration

Generation is controlled by requesting a target distribution: an error-type mix, a class balance, a structural target, but an LLM does not honour every category equally, so what a run delivers is measurably different from what it requested. Calibration measures that gap and corrects the request, and the sprint's main design decision is *where* that correction happens. Correcting it inside a scored session would make the session's repeated runs non-independent, turning the mean and standard deviation the framework reports for every metric into a mix of genuine generator noise and the controller's own settling behaviour. Calibration therefore runs as a separate, unscored phase before the experiment: it produces a reusable artifact, and the scored session then runs unchanged at that fixed setting.

The calibration loop itself is iterative: round zero requests exactly the target distribution with no correction; each subsequent round generates a trial batch, measures the delivered distribution against the target with Jensen-Shannon divergence, and applies a damped update before the next round. Because the correction is purely numerical, no wording is ever added to a generation prompt, which keeps calibration from teaching the generator what the fidelity checks look for.

What calibration actually steers is task- and cell-specific. Most imposed cells correct the same type and count distribution that seeds the prompt; Spam Detection additionally corrects its class balance in closed form, since a model refuses to write spam far more often than it refuses a benign paraphrase; GEC's forward + seeded cell has no injectable distribution at all, since the generator chooses its own error type, so calibration instead reweights which seeds are drawn.

2.3 Generation Strategies

Every task declares a generation *strategy*: **corruption** for GEC and Sentiment Analysis, **class_conditional** for Spam Detection, **structured** for Taxonomy Induction, which fixes the shape of one generated sample: a corrupted sentence, a labelled message, or a whole taxonomy. Independently of that shape, every task reads the same two generation settings, which together select one of four *cells*:

- **mode** (**forward** or **inverse**) controls where the annotation comes from. **inverse** draws it independently and *imposes* it on the source; **forward** *inherits* it from the source instead of imposing anything new.

- **seedless** (true or false) controls whether real benchmark text ever reaches the generation prompt. When false (*seeded*), a real example is passed as a seed; when true (*seedless*), no real example is used, and a content specification sampled from the benchmark profile (Section 2.1) drives generation instead.

	Seeded (seedless: <code>false</code>)	Seedless (seedless: <code>true</code>)
Forward	Inherit the annotation from a real seed.	Inherit nothing; generate directly from a profile-sampled content specification.
Inverse	Impose an independently drawn annotation on a real seed.	Impose an independently drawn annotation on a synthetic, profile-sampled carrier.

Table 1: The four generation cells, independent of task shape.

The concrete meaning of the imposed or inherited annotation depends on the task shape. For **corruption** (GEC, Sentiment Analysis), forward + seeded rewrites a real sentence and lets the generator pick its own error type, while inverse cells corrupt a real or synthetic clean sentence according to an empirical error distribution profiled from the benchmark. For **class_conditional** (Spam Detection), a target label is drawn from a class balance and a message of that class is synthesised; every class is generated text, so a classifier cannot separate classes on a “written by an LLM” artifact. For **structured** (Taxonomy Induction), one sample is a complete synthetic taxonomy: seeded cells anonymise a real subtree and ask the model to re-verbalise it into a new domain, while seedless cells sample a structural target from the profile and, in inverse mode, drive a bounded feedback loop toward it.

2.4 Module decoupling

Every stage of the pipeline is its own entry point, works uniformly across all four tasks, and reads or writes the same on-disk session format, so stages can be run independently, on different machines, and at different times, without re-running earlier stages.

3 Tasks

Both tasks added this sprint plug into the same task interface as GEC and Spam Detection: a task subclasses a common base class, declares its generation strategy, error types or class labels, prompts, evaluators and models, and is then driven by the same pipeline, profiling, calibration and judge machinery described in Section 2, with no task-specific pipeline code.

Stage	Independent entry point
Benchmark profiling	<code>framework.profile_dataset</code>
Calibration	<code>framework.calibrate</code>
Generation only	<code>framework.main --generate-only</code>
Evaluation only	<code>scripts.rescore_session</code>
Generation and evaluation	<code>framework.main</code>
Plotting	<code>framework.plotting</code>
Multi-model comparison	<code>scripts.compare_models</code>
Cross-session analysis	<code>scripts.analyze_results</code>

Table 2: Independently runnable pipeline stages.

3.1 Sentiment Analysis

Sentiment Analysis uses the **corruption** strategy: the generator rewrites a real tweet by injecting a sentiment transformation (for example negation insertion, sarcasm injection, or intensity reduction), which implies a deterministic ground-truth label rather than requiring a separate annotation step. The benchmark is a 150-row subset of TweetEval’s (Barbieri et al. 2020) sentiment test split, prepared once per clone from a fixed recipe. Each transformation maps to one of three labels: negation- and sarcasm-style transformations to NEGATIVE, their positive counterparts to POSITIVE, and intensity reduction to NEUTRAL.

All four generation cells are supported. The shipped baseline is forward + seeded, where a real tweet is rewritten and the model picks its own transformation; the inverse cells instead draw the transformation from the real label balance and impose it, either on a real tweet (seeded) or on a synthetic carrier built from the benchmark profile (seedless). Two fine-tuned three-class classifiers are evaluated as task models: a BERTweet-based model and a multilingual DistilBERT model. Evaluation reports accuracy plus macro-averaged precision, recall and F1 across the three classes.

3.2 Taxonomy Induction

Taxonomy Induction uses the **structured** strategy: one generated sample is an entire synthetic taxonomy: a domain, a set of class identifiers, and a set of subclass axioms between them. Whole-artifact generation needs a much larger token budget and timeout than a sentence rewrite, since a reasoning model spends part of that budget thinking before it emits the artifact.

The benchmark is the Protege Pizza ontology (OWL API contributors 2026), converted once from OWL/RDF into a normalised JSONL file that keeps named classes, direct subclass edges, and the named parents of equivalent-class intersection definitions, while excluding anonymous restrictions, unions, and any reasoner-inferred structure. Seeded cells sample a real subtree, anonymise every class name, and ask the model to re-verbalise it into a new domain while preserving every relation; the response is checked

against a gold structure computed directly from the sampled subtree (inherited as-is for forward, edited toward a profile-sampled target for inverse) by exact positional match rather than graph isomorphism, since ontologies are directed acyclic graphs and a canonicalisation could accept a structurally different graph as equivalent. Seedless cells instead sample a purely structural target from depth and branching distributions and, in inverse mode, drive a bounded feedback loop of at most one extra regeneration round toward it; the loop compares the draft’s measured structure against the target and requests a correction, without ever exposing real class names or edges.

Three task models are evaluated: two non-LLM structural baselines (a longest-suffix lexical heuristic, and a “star” baseline that places every class under one root) and the LLM scored separately on the un-anonymised classes and domain. Evaluation is exact ordered child-parent precision, recall and F1, with malformed or unknown-class predictions tracked as diagnostics rather than silently scored as wrong. The calibration phase (Section 2.2) applies only to the inverse + seedless cell.

4 Ablation

4.1 Setup

The ablation study is designed to justify, for every task, a default configuration: which generation cell to use, whether to calibrate it, which generation model to use, and whether to judge its output. It proceeds in four phases, each fixing what the previous phase established and varying one further factor. Every phase reuses the independently runnable stages described in Section 2.4, so later phases can be run, and any phase re-scored or re-plotted, without repeating earlier ones.

Phase	Cells run	Generation model(s)	Calib.	Judge
A - baseline sweep	all four, per task	one shared model	off	off
B - calibrated best cell	best cell from A, per task	same shared model	on	off
C - multi-model	best cell from A, per task	six generators (one comparison run per generator)	on	off
D - judged	best cell from A, per task	same shared model	on	on

Table 3: The four ablation phases.

Phase A runs all four generation cells independently for every task, with the judge and calibration both off and the real-benchmark baseline enabled, so the only factor varying across a task’s four sessions is the cell itself. Each session repeats three runs of 150 generated samples for GEC, Spam Detection and Sentiment Analysis, or three runs of ten whole taxonomies for Taxonomy Induction.

Phase B takes, for each task, the cell that Phase A shows to fit the real benchmark most closely, and re-generates it with a calibration artifact applied, with everything else unchanged. That artifact is produced once with the calibration entry point before this phase runs, so Phase B measures how much of the remaining gap to the real benchmark calibration closes on top of an already-chosen cell, rather than which cell to choose in the first place.

Phase C fixes every task at its Phase A/B cell and instead varies the generation model, using the multi-model comparison entry point to run that same cell once per generator, across six models, including the one shared model used in Phases A, B and D. This isolates how much of an effect attributed to a task or a cell elsewhere in the study is really specific to one generator.

Phase D takes each task’s Phase A/B cell and switches on the optional LLM-as-judge quality gate, which filters generated samples before task-model evaluation and archives every rejected sample together with the judge’s verdict. This is the final sweep’s own ablation condition: judged and unjudged sessions of the same cell differ only in whether a semantic quality filter sits between generation and evaluation, isolating what that filter changes on top of the mechanical checks every cell already applies.

4.2 Results

All sessions below use the framework’s standard, shipped sample sizes: three runs of 150 generated samples for GEC, Spam Detection and Sentiment Analysis, three runs of ten whole taxonomies for Taxonomy Induction, so these are the results the setup in Section 4.1 was designed to produce.

Phase A: which cell is realistic? Figure 1 shows, per task, the generated-vs-real gap on each task’s headline metric across all four cells (GEC and Spam average over every task model; Sentiment averages over its two classifiers; Taxonomy shows its LLM task model only, since its two structural baselines are not meaningful to average in). The pattern differs sharply by task shape: for GEC, inverse + seeded comes far closer than the rest (0.33 generated vs. 0.23 real ERRANT F0.5) though still clearly inflated; every other GEC cell inflates scores by 0.44-0.56, the largest gaps in the figure. For Spam, the opposite holds: forward + seeded is already almost exactly real (0.98 vs. 0.97 F1), while both inverse cells depress every classifier substantially (down to 0.66-0.69 mean F1). Sentiment sits closer to real everywhere, with forward + seedless the furthest off. Taxonomy is the only task whose gap changes sign across cells rather than just its size.

Table 4 summarises the cell each task’s team member picked as “best” in Phase A, that cell’s own fidelity gap, what calibration (Phase B) did to the distribution it targets, and what fraction of samples the judge (Phase D) removed for that cell.

Phase B: calibration. Two of four tasks converge at round 0 - the uncalibrated request already matches what the benchmark delivers within tolerance, so calibration is a no-op for Sentiment and Taxonomy (by construction, since its feedback loop already drives fidelity per sample). GEC improves modestly at round 1 and then gets *worse* at rounds 2-3 before the loop’s “best round wins” rule discards those; downstream F0.5 moves by about 0.02 (0.33 to 0.31). Spam’s type distribution was already well matched

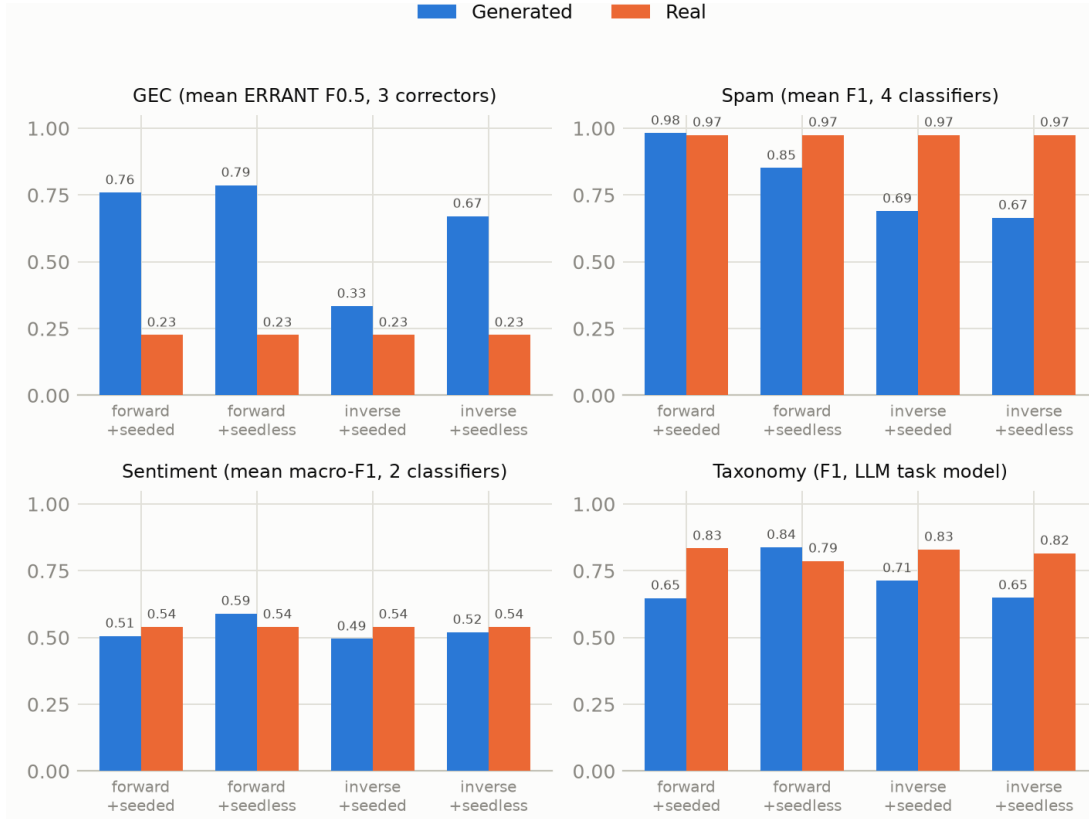


Figure 1: Phase A: generated vs. real score on each task’s headline metric, across all four generation cells. GEC and Spam bars average over every task model; Sentiment over both classifiers; Taxonomy shows the LLM task model only.

from round 0 (JSD 0.006); its count distribution is the real problem, and three correction rounds only partly close it (JSD 0.30 $\rightarrow$ 0.16) while the controller’s own requested count distribution runs away toward “one signal per message” (88% requested at round 3 vs. 3% actually measured).

Phase C: does the generation model matter? For GEC and Sentiment, swapping the generation model shifts the headline metric by a similar amount (GEC F0.5 0.28-0.37, Sentiment macro-F1 0.70-0.79 across the six generators), and the shared default (`minimax-m3`) is mid-pack in both, not closest to real: `xiaomi/mimo-v2.5` comes closest for GEC (0.28), `openai/gpt-5.6-luna` for Sentiment (0.70). Spam is largely insensitive to generator choice for its three strongest classifiers, and only the weakest one moves noticeably (0.92-0.98 F1). Taxonomy is the exception: as Figure 2 shows, the LLM task model scores 0.64 F1 on taxonomies it generated itself, but only 0.06-0.51 F1 on taxonomies from every other generator, at the same cell and sample size. The two structural baselines (lexical, star) stay near zero for every generator (star 0.02-0.03; lexical 0.00-0.05, faintly tracking the same order) - the driver is each generator’s own

Task	Best cell	Phase A gap (gen vs. real)	Calibration JSD, round 0 $\rightarrow$ selected	Judge drop rate
GEC	inverse + seeded	+0.11 F0.5 (0.33 vs. 0.23)	type: 0.17 $\rightarrow$ 0.13 (round 1)	19%
Spam	forward + seeded	+0.01 F1 (0.98 vs. 0.97)	count: 0.30 $\rightarrow$ 0.16 (round 3)	51%
Sentiment	inverse + seed- less	-0.02 macro-F1 (0.52 vs. 0.54)	type: 0.011 $\rightarrow$ 0.011 (round 0)	67%
Taxonomy	inverse + seed- less	-0.17 F1 (0.65 vs. 0.82)	type: $\approx 0 \rightarrow \approx 0$ (round 0)	74%

Table 4: Per-task summary: the picked cell’s Phase A fidelity gap, what calibration corrected in Phase B, and the judge’s Phase D drop rate.

class-naming convention, not the framework’s checks (see Section 5).

Phase D: judging. Drop rates (Table 4) scale with how much room a task’s generation has to go wrong: GEC’s fairly constrained corruption prompt loses about a fifth of samples, while Sentiment’s and Taxonomy’s more open-ended generation loses roughly two thirds to three quarters. For GEC and Spam, the surviving samples score close to their unjudged Phase A/B numbers, i.e. the judge filters without materially changing the difficulty of what is left. Sentiment is more mixed: the stronger classifier ticks up slightly (0.68–0.72 without judging to 0.71 judged) while the weaker one drops sharply (0.35 to 0.21) on a judge-shrunk sample of only around 50 items - too small to tell a real judge bias apart from ordinary sampling noise. Taxonomy’s judged F1 (0.73, std 0.18) is nominally the best any inverse+seedless session reached, but from only 2-5 surviving taxonomies per run; at that count a single artifact can swing the mean by tens of points, so this number should not be read as an improvement over Phase A/B without a larger judged run to confirm it.

5 Analysis

All together, the four ablation phases point to one central lesson: whether generated data is a realistic stand-in for the real benchmark depends on the *cell*, and the reason differs by task shape rather than being a single framework-wide property.

For GEC, a **corruption** task, realism tracks the *seed*, not the error-type distribution: inverse + seeded and inverse + seedless apply the same empirically profiled error mix, yet only the seeded version comes anywhere close to real (0.33 vs. 0.23 F0.5); the seedless version is nearly as inflated as either forward cell (0.67, against 0.76-0.79 for forward). For Spam, a **class_conditional** task, the opposite mechanism produces the opposite ranking: forward inherits a seed’s own label and lightly rewrites it, which stays close to how real spam and ham actually read, while inverse imposes a label independently of

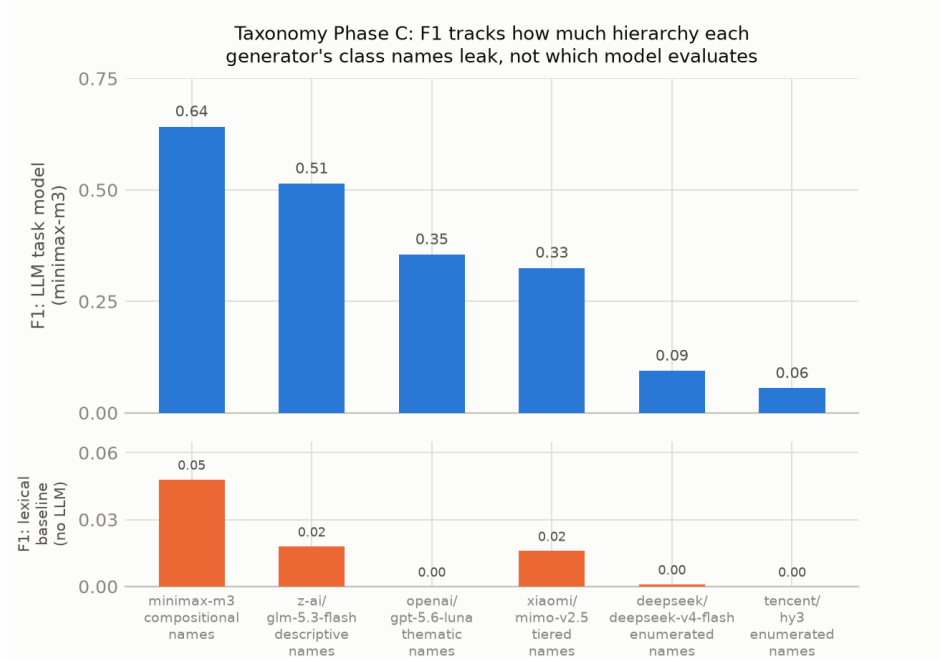


Figure 2: Taxonomy Phase C: F1 by generation model, for the LLM task model (top) and the lexical baseline (bottom), with each generator’s class-naming style. Both rank generators in the same order, evidence that naming informativeness, not which model does the evaluating, drives the spread.

style and produces something closer to a stress test (a spam-topic message paraphrased clean, or a benign message injected with spam signals) than a realistic message. Neither result says one axis is “better” in general; each says that *for this task shape*, one axis’s job (inherit vs. impose) happens to align with what real data looks like, and the other does not. That is exactly the kind of fact a per-task ablation is needed to establish, since it could not have been predicted from the cell definitions alone.

Calibration’s contribution across all four tasks is real but small. Two tasks (Sentiment, Taxonomy) never needed it in this run, since their round-0 request already matched what the benchmark delivers, for different reasons in each case (Sentiment’s transformation mix is close to the real label balance already; Taxonomy’s own inverse-mode feedback loop corrects toward the target per sample, leaving little for a second, cross-sample correction to fix). GEC gained a little and Spam gained more, but Spam’s remaining gap is bounded by a ceiling the loop cannot correct past: the model will not write as many single-signal messages as calibration ends up requesting, no matter how many rounds are spent asking. This suggests calibration is best understood as insurance against a specific, checkable failure mode (a model’s own categorical bias) rather than a general fidelity dial, and that the count-distribution ceiling for Spam is worth investigating directly (e.g. by loosening `class_balance` or the signal-mix request itself) rather than by adding calibration rounds.

The clearest new finding from this sprint’s work is Taxonomy’s Phase C score spread, and inspecting the generated class names explains it rather than leaving it a mystery. Generators differ enormously in how much hierarchy information they put directly into class names: minimax-m3’s own taxonomies name classes compositionally (`ThinCrustFoundation` $\rightarrow$ `CrispyThinBase` $\rightarrow$ `CrispyPlainVariant`), so an evaluator given only the domain and class list can recover much of the structure from the strings themselves; deepseek and tencent instead emit largely opaque, enumerated names (`Pizza_1`, `Pizza_2`, ...; `CheeseLattice_011` through `034`) that carry almost no signal. The lexical baseline’s scores track the same order at a much smaller scale (0.05 on minimax’s own taxonomies vs. 0.00 on deepseek’s and tencent’s), which rules out the alternative explanation that minimax the evaluator simply recognises its own writing style: a heuristic with no model behind it shows the same ranking. The run diagnostics also rule out a parsing bug, malformed, invalid and unknown-class prediction counts are 0 for every generator, so these are real scores, not an artifact. The result is still a genuine limitation: this cell’s evaluation does not control for naming informativeness across generators, so Phase C’s taxonomy comparison measures that confound at least as much as generation quality. Comparing generation models on this cell in future should anonymise class names (or require structurally-neutral identifiers) before scoring, not evaluate the raw generated names.

Finally, the judge’s Phase D drop rates (19-74%) show that a semantic quality filter is doing substantially more work for the more open-ended tasks than for GEC’s constrained rewrite. Where enough samples survive to average over (GEC, Spam), judging changes little; where they do not (Sentiment’s weaker classifier, all of Taxonomy), the resulting numbers carry enough sampling noise that they should be treated as preliminary rather than conclusive until the affected cells are re-run at a larger sample size with the judge enabled.

6 Conclusion

This sprint set out to turn a working but single-purpose GEC/Spam pipeline into a general framework, and to ask, systematically, how much of the GET methodology’s promise, a synthetic benchmark that stands in for a real one without the contamination risk, actually holds up under measurement. The ablation study answers that question with a qualified yes, most clearly for Spam: forward + seeded gets within a point of the real benchmark using nothing but a profiled distribution and an LLM generator. GEC’s best cell, inverse + seeded, does not reach that bar on the stricter ERRANT F0.5 metric (0.33 vs. 0.23 real), but it is still the only GEC cell in the same order of magnitude as real, against gaps of 0.44-0.56 for the other three. Both results say the cell choice, not the framework as a whole, is what determines whether generation gets close to real difficulty.

The same study is equally clear about where that promise is not yet realised. Realism is cell-specific and task-shape-specific rather than a property of the framework as a whole, so choosing the right cell per task is not optional set-up work but the main lever

available. Calibration, in its current form, offers only a small correction on top of that choice, bounded by ceilings (a model’s own categorical bias, a small seed pool) that more rounds cannot cross. The choice of generation model matters more for some tasks (GEC, Taxonomy) than others (Spam), and Taxonomy Induction in particular is not yet a fair cross-model benchmark: its inverse+seedless cell measurably favours whichever model also serves as the task model, and its small per-run sample size (ten taxonomies) leaves both its calibration and its judged results more exposed to noise than the sentence-level tasks. An optional LLM judge can filter a large and task-dependent share of generated samples for a quality gain that is hard to distinguish from sampling noise at current sample sizes.

None of this undermines the core premise: live, generated evaluation data can be as discriminating as, and sometimes harder than, a fixed real benchmark, and doing so is now a matter of picking the right configuration per task. What remains is narrower and more concrete: run tasks judged at a larger sample size before trusting their numbers, and treat calibration as a targeted fix for a diagnosed bias rather than a default step to enable everywhere.

References

- Almeida, T. A., J. M. Gómez Hidalgo, and A. Yamakami (2011). Contributions to the study of SMS spam filtering: New collection and results. In *Proceedings of the 11th ACM Symposium on Document Engineering (DocEng ’11)*, Mountain View, California, USA, pp. 259–262. ACM.
- Barbieri, F., J. Camacho-Collados, L. Espinosa Anke, and L. Neves (2020). TweetEval: Unified benchmark and comparative evaluation for tweet classification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Online, pp. 1644–1650. Association for Computational Linguistics.
- Bryant, C., M. Felice, Ø. E. Andersen, and T. Briscoe (2019). The BEA-2019 shared task on grammatical error correction. In *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications*, Florence, Italy, pp. 52–75. Association for Computational Linguistics.
- OWL API contributors (2026). The pizza ontology. <https://github.com/owlcs/pizza-ontology>, commit 4922ecb. Originally developed for the University of Manchester’s OWL/Protégé tutorials.
- Yannakoudakis, H., T. Briscoe, and B. Medlock (2011). A new dataset and method for automatically grading ESOL texts. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, Portland, Oregon, USA, pp. 180–189. Association for Computational Linguistics.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?
Claude	Rephrasing	Throughout	+
Gemini	Summarization of related papers	Earlier sprints	+
Claude Code	Coding	Throughout	+

Unterschrift

Mannheim, den XX. XXXX 2026